```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Student {
  int regNo;
  char name[50];
  struct Student *next;
};

struct Student *head = NULL;

/* a. Add a student */
void addStudent() {
  struct Student *newNode, *temp;

  newNode = (struct Student *)malloc(sizeof(struct Student));

  if (newNode == NULL) {
    printf("Memory allocation failed!\n");
    return;
  }

  printf("Enter Registration Number: ");
  scanf("%d", &newNode->regNo);

  printf("Enter Student Name (use _ for spaces): ");
  scanf("%49s", newNode->name);

  newNode->next = NULL;

  if (head == NULL) {
    head = newNode;
  } else {
    temp = head;

    while (temp->next != NULL) {
      temp = temp->next;
    }

    temp->next = newNode;
  }

  printf("Student added successfully.\n");
}
```

```
/* b. Remove a student */
void removeStudent() {
   struct Student *temp = head;
   struct Student *prev = NULL;
   int regNo;

   printf("Enter Registration Number to remove: ");
   scanf("%d", &regNo);

   while (temp != NULL && temp->regNo != regNo) {
      prev = temp;
      temp = temp->next;
   }

   if (temp == NULL) {
      printf("Student not found.\n");
      return;
   }

   if (prev == NULL) {
      head = temp->next;
   } else {
      prev->next = temp->next;
   }

   free(temp);
   printf("Student deleted successfully.\n");
}

/* c. Display the list */
void displayStudents() {
   struct Student *temp = head;

   if (head == NULL) {
      printf("No students registered.\n");
      return;
   }
   printf("\nRegistration Number\tName\n");

   while (temp != NULL) {
      printf("%d\t\t\t%s\n",
           temp->regNo, temp->name);
      temp = temp->next;
   }
}
```

```
/* d. Count total participants */
void countStudents() {
  struct Student *temp = head;
  int count = 0;

  while (temp != NULL) {
    count++;
    temp = temp->next;
  }

  printf("Total participants: %d\n", count);
}

/* e. Sort by Registration Number */
void sortStudents() {
  struct Student *i, *j;
  int tempReg;
  char tempName[50];

  if (head == NULL) {
    printf("List is empty.\n");
    return;
  }

  for (i = head; i->next != NULL; i = i->next) {
    for (j = i->next; j != NULL; j = j->next) {
      if (i->regNo > j->regNo) {
        tempReg = i->regNo;
        i->regNo = j->regNo;
        j->regNo = tempReg;

        strcpy(tempName, i->name);
        strcpy(i->name, j->name);
        strcpy(j->name, tempName);
      }
    }
  }

  printf("Students sorted successfully.\n");
}
```

```
/* f. Reverse the list */
void reverseList() {
  struct Student *prev = NULL;
  struct Student *current = head;
  struct Student *next = NULL;

  while (current != NULL) {
    next = current->next;
    current->next = prev;
    prev = current;
    current = next;
  }

  head = prev;

  printf("List reversed successfully.\n");
}

/* Release allocated memory */
void freeList() {
  struct Student *temp;

  while (head != NULL) {
    temp = head;
    head = head->next;
    free(temp);
  }
}

int main() {
  int choice;

  do {
    printf("\n--- Workshop Enrollment System ---\n");
    printf("1. Add Student\n");
    printf("2. Remove Student\n");
    printf("3. Display Students\n");
    printf("4. Count Participants\n");
    printf("5. Sort Students\n");
    printf("6. Reverse List\n");
    printf("0. Exit\n");

    printf("Enter your choice: ");
    if (scanf("%d", &choice) != 1) {
      freeList();
```

```
            return 1;
        }

        switch (choice) {
            case 1:
                addStudent();
                break;

            case 2:
                removeStudent();
                break;

            case 3:
                displayStudents();
                break;

            case 4:
                countStudents();
                break;

            case 5:
                sortStudents();
                break;

            case 6:
                reverseList();
                break;

            case 0:
                printf("Exiting program.\n");
                break;

            default:
                printf("Invalid choice. Try again.\n");
        }

    } while (choice != 0);

    freeList();

    return 0;
}
```